AgroGest

Guia de *defesa.*

O que o docente provavelmente vai perguntar, as respostas de acordo com o que está implementado, onde está cada coisa no código, o guião da demonstração ao vivo e o que convém saber antes do dia.

01 O AgroGest em 1 minuto
02 Mapa do código
03 Arquitectura e MVC
04 Regras de negócio (RN01–RN06)
05 Validações e usabilidade
06 Filtros, sessões e segurança
07 Persistência
08 Deploy e systemd
09 Perguntas de "e se..."
10 Guião da demonstração
11 Comportamentos a conhecer
12 Decisões e pontos fracos
13 Lista para o dia

Projecto
AgroGest — Sistema de Gestão Agrícola

Disciplina
Sistemas Distribuídos · ISCTEM

Testes
37 unitários · 80 verificações de ponta a ponta

01

O AgroGest em 1 minuto

Para abrir a apresentação (≈ 1 minuto, sem ler):

> "O AgroGest é uma aplicação web em **Java 17 com Servlets e JSP**, publicada no **Tomcat 10.1** na porta 8081, para um agricultor de Nampula. O **administrador** gere fornecedores, produtos, campanhas, cultivos e colheitas, e acompanha vendas e relatórios. O **cliente** regista-se, compra no catálogo e recebe uma **factura com IVA de 16%**. Não há base de dados: os dados vivem em **listas em memória sincronizadas** e são **serializados para ficheiro** com escrita atómica. A venda é **transaccional**: ou desconta o stock de todos os itens, ou de nenhum. Seguimos **MVC** com pacotes model, repo, service, servlet, filter, listener e util, e temos 37 testes automáticos."

Números para ter na ponta da língua

Classes Java (sem testes)	55, em 7 pacotes	Páginas JSP	20 + fragmentos .jspf
Testes JUnit	37 (29 regras + 4 validação + 4 formatação)	Teste de ponta a ponta	80 verificações
Porta / contexto	8081 · `/agrogest`	Formulários POST com token CSRF	24
Hash de passwords	PBKDF2-HMAC-SHA256, 65 536 iterações	IVA	16% sobre o subtotal
Dados de exemplo	3 fornecedores · 3 produtos · 5 clientes · 4 vendas · 1 campanha	1.ª factura da demo	FT-2026-1

02

Mapa do código — "onde está isso?"

SE PERGUNTAREM POR...	ABRA
Venda atómica, stock, validade	`service/VendaService.java` → `registarVenda`
Anular venda / ajustar carrinho	`VendaService.anularVenda` / `ajustarAoStock`
Controlo de acesso (sessão, 403)	`filter/AuthFilter.java`
Protecção CSRF	`filter/CsrfFilter.java`
Listas em memória e IDs (AtomicInteger)	`repo/DataStore.java`
Gravar / restaurar ficheiro	`repo/Persistencia.java` + `listener/AppListener.java`
Passwords	`util/PasswordUtil.java`
Tentativas de login	`service/TentativasLogin.java`
Telefone, nome, decimais	`util/Validacao.java`
Listas fechadas (províncias, unidades, culturas)	`util/Opcoes.java`
Tipo de fornecedor → categorias	`model/TipoFornecedor.java`
Campanhas, cultivos, colheitas	`service/AgricolaService.java`
Fornecedores e produtos	`service/CatalogoService.java`

SE PERGUNTAREM POR...	ABRA
Relatórios e ordenação	`service/RelatorioService.java`
Erro junto ao campo / repor formulário	`servlet/BaseServlet.executar` + `js/app.js`
Rotas	anotação `@WebServlet` no topo de cada servlet
Testes	`src/test/java/…/RegrasNegocioTest.java`

03

Arquitectura e MVC

P. **Explique a arquitectura. Onde está o Model, a View e o Controller?**

R. **Model**: POJOs Serializable em `model/` (atributos privados, construtores, getters/setters). **Controller**: servlets em `servlet/`, que lêem os parâmetros, chamam o serviço e encaminham para a JSP. **View**: JSP + JSTL em `WEB-INF/views/`. As regras de negócio estão nos **services** e os dados no **repo**, por isso os servlets ficam finos.

P. **Porque estão as JSP dentro de WEB-INF?**

R. Para não serem acessíveis por URL directo. Só são mostradas por `forward` a partir de um servlet, depois de passar pelos filtros e de os dados estarem preparados.

P. **Diferença entre forward e redirect? Onde usaram cada um?**

R. `forward` é interno ao servidor (o URL não muda): para mostrar páginas. `redirect` pede ao browser um novo GET: depois de cada POST (**Post/Redirect/Get**), para que um F5 não repita a compra.

P. **Para que serve a BaseServlet?**

R. Evita repetir código: `render`, `redirect`, acesso ao carrinho e `executar`, que corre a acção, mostra a mensagem de sucesso ou, numa `NegocioException`, mostra o erro, guarda os valores escritos e o nome do campo errado para a página seguinte.

P. **O que é uma NegocioException?**

R. Uma excepção nossa para violações de regras. Tem a mensagem para o utilizador e, opcionalmente, o **campo** com erro (ex.: `"telefone"`), que a vista usa para assinalar esse campo.

P. **Porque usaram listeners e filtros anotados em vez do web.xml?**

R. O enunciado pede `@WebServlet`, `@WebFilter` e `@WebListener`. O web.xml só tem configuração geral: codificação UTF-8, sessão, páginas de erro e o prelúdio das taglibs.

04

Regras de negócio (RN01–RN06)

P. **Como garantem que a venda é atómica (tudo ou nada)? — RN-b**

R. Em `registarVenda`, dentro de `synchronized (DataStore.get())`: **fase 1** valida todos os itens (existe, quantidade válida, dentro da validade, stock suficiente) sem alterar nada; se houver erros, lança a excepção. **Fase 2** só então desconta o stock e cria a venda — daí em diante não há falhas possíveis.
Teste: `vendaEAtomicaTudoOuNada`

P. **E se dois clientes comprarem o último saco ao mesmo tempo? — RN-a**

R. Todas as escritas usam `synchronized` sobre o mesmo objecto, por isso as compras são feitas uma de cada vez; a segunda já vê o stock actualizado e falha.
Teste: `comprasConcorrentesNuncaVendemMaisDoQueOStock` — 20 compras em paralelo sobre 10 unidades → só 10 passam e o stock fica em 0.

P. **Porque precisam de synchronized se já usam Collections.synchronizedList?**

R. A lista sincronizada protege cada operação isolada (um `add`). Uma venda são várias operações seguidas (ler, validar, descontar, adicionar) — precisam de um trinco à volta de todas.

P. **O stock é validado só no fim?**

R. Não, em três sítios: no **formulário** (o campo tem `max` = stock − o que já está no carrinho), ao **adicionar/actualizar** o carrinho e outra vez na **confirmação**, dentro do trinco. A última é a que garante a regra; as outras evitam erros ao utilizador.

P. **Como impedem um cliente de ver a factura de outro? — RN-c**

R. `VendaService.podeVer`: só o admin ou o dono. O `FacturaServlet` devolve **403**; mudar o `?id=` no URL não chega.

P. **Como funcionam os IDs e o número da factura? — RN-f**

R. Um `AtomicInteger` por entidade no `DataStore`. A factura tem sequência própria **por ano**: `FT-<ano>-<n>`. As vendas de exemplo são de 2025, por isso a 1.ª compra da demo é a FT-2026-1.

P. **Que outras regras impedem remoções?**

R. Campanha com cultivos, cultivo com colheitas e fornecedor com produtos não podem ser removidos. Campanha fechada não aceita cultivos nem colheitas.

05

Validações e usabilidade NOVO

P. O cliente pode comprar mais do que existe no stock?

R. Não. O campo de quantidade tem máximo, o servidor recusa ao adicionar ("Indisponível: só há N … em stock (já tem X no carrinho)") e volta a verificar na confirmação. Há ainda um limite de 10 000 unidades por linha e só se aceitam números inteiros.

P. E produtos fora de validade?

R. Não se vendem: são bloqueados ao adicionar e na confirmação (a regra usa a data da venda). No catálogo aparecem a cinzento com "Fora de validade". O painel do admin lista os que expiram nos próximos 30 dias.

P. Onde usaram caixas de selecção?

R. Província (11 províncias, no fornecedor, no registo e no perfil), unidade do produto, categoria, tipo de fornecedor, campanha e cultivo, e **cultura** (13 culturas + "Outra…" que abre um campo de texto), com **variedades sugeridas** conforme a cultura. O fornecedor do produto só mostra os que vendem a categoria escolhida.

P. As listas são validadas só no browser?

R. Não. Tudo o que vem do browser é validado **no servidor** (`Opcoes`, `Validacao`, services). As validações HTML (`required`, `max`, `pattern`) só melhoram a experiência — qualquer pessoa as consegue contornar.

P. Que regras têm as datas agrícolas?

R. Campanha: fim depois do início, no máximo 24 meses, nome único; se se sobrepuser a outra, o sistema **avisa** (não bloqueia). Plantio dentro das datas da campanha. Colheita depois do plantio e até 6 meses após o fim da campanha. Editar uma campanha não pode deixar cultivos de fora.

P. E o telefone e o nome?

R. Telefone moçambicano: telemóvel 82–87 (9 dígitos) ou fixo 2x (8 dígitos), com ou sem +258, guardado sempre como "+258 84 123 4567". Nome do cliente: nome e apelido, só letras, espaço, hífen e apóstrofo.

P. O que acontece quando o utilizador erra um campo?

R. A mensagem aparece no topo **e por baixo do campo**, que fica a vermelho e recebe o foco; os valores escritos são repostos (excepto as palavras-passe). Funciona em todos os formulários.

P. Como evitam uma compra duplicada por duplo clique?

R. O botão desactiva-se depois do primeiro envio ("A processar…"). E mesmo que chegassem dois pedidos, o segundo encontraria o carrinho vazio e falharia — a regra está no servidor.

P. **E se houver um engano numa venda?**

R. O admin pode **anular**: as quantidades voltam ao stock, a factura fica marcada "ANULADA" (mantém o número, como numa contabilidade real) e deixa de contar na facturação e nos relatórios. Não é possível anular duas vezes.

P. **Que outras funcionalidades acrescentaram além do enunciado?**

R. Pesquisa por nome (sem ligar a acentos), paginação, edição de fornecedores e campanhas, reposição rápida de stock (+10/+50/+100), "ajustar o carrinho ao stock", página de perfil do cliente, anulação de vendas, alerta de validade e página inicial pública.

06

Filtros, sessões e segurança

P. O que faz o AuthFilter? — RN-d e RN-e

R. Rotas do cliente sem sessão → redirect para `/login`. `/admin/*` com um não-admin → **403**. Carrinho, compra, histórico e perfil com o admin → 403 (o admin não compra). Em cada pedido relê a conta: um cliente desactivado perde o acesso logo.

P. O que é CSRF e como se protegeram? NOVO

R. É um site malicioso fazer o browser da vítima enviar um formulário para o nosso (ex.: uma compra) usando a sessão dela. O `CsrfFilter` cria um **token aleatório por sessão** que vai num campo escondido `_csrf` de todos os 24 formulários POST; um POST sem o token certo é recusado (403). Há ainda o cookie `SameSite=Lax` e `HttpOnly`.

P. Como guardam as palavras-passe?

R. **PBKDF2-HMAC-SHA256**, 65 536 iterações, sal aleatório de 16 bytes, guardado como `iterações:sal:hash`. A comparação usa `MessageDigest.isEqual` (tempo constante). Um SHA-256 simples seria rápido de mais para resistir a força bruta.

P. E ataques de adivinhar a palavra-passe? NOVO

R. 5 falhas seguidas para o mesmo email bloqueiam-no durante 1 minuto (`TentativasLogin`), mesmo que a 6.ª tentativa esteja certa.

P. O que é fixação de sessão?

R. O atacante impõe um ID de sessão à vítima e espera que ela entre. No login fazemos `request.changeSessionId()`, por isso o ID muda.

P. E XSS?

R. Tudo o que o utilizador escreve é mostrado com `<c:out>` ou `fn:escapeXml`, que escapam o HTML. O URL da foto só aceita https:// ou caminhos da aplicação (bloqueia `javascript:`).

P. Onde fica o carrinho?

R. Na `HttpSession`, como objecto `Carrinho` (produtoId → quantidade). Perde-se se o Tomcat reiniciar — é dado de sessão, como o enunciado pede.

07

Persistência

P. Como guardam os dados sem base de dados?

R. Serialização Java (`ObjectOutputStream`) de todo o `DataStore` para `agrogest.ser`. O `AppListener` restaura no arranque (`contextInitialized`) e grava ao encerrar (`contextDestroyed`). Também há gravação e eliminação manual em *Persistência*.

P. E se o Tomcat for abaixo a meio da gravação?

R. Gravamos primeiro em `.tmp`, forçamos para o disco (`fsync`) e só depois fazemos `Files.move` com `ATOMIC_MOVE`. A versão anterior fica em `.bak`; se o principal estiver corrompido, carrega-se o `.bak`.
Teste: `persistenciaUsaBackupSeOFicheiroPrincipalEstiverCorrompido`

P. Para que serve o serialVersionUID? E se mudarem as classes?

R. Identifica a versão da classe. Acrescentar campos (como fizemos com `provincia` e `anulada`) é compatível: os ficheiros antigos carregam com os campos novos vazios.

08

Deploy e systemd

P. Como fazem o deploy?

R. `./mvnw package` gera `target/agrogest.war`; copia-se para `webapps/`; porta 8081 no `conf/server.xml`. Em Linux, `deploy/instalar.sh` instala Java 17 e Tomcat 10.1 e regista o `tomcat.service`.

P. O que faz o systemd aqui?

R. Arranca o Tomcat com o sistema, reinicia-o se falhar (`Restart=on-failure`) e permite `systemctl start/stop/status`. O `stop` envia SIGTERM → encerramento ordenado → é aí que os dados são gravados (`TimeoutStopSec=60` dá tempo para isso).

P. Porquê Tomcat 10.1 e não 9?

R. O 10.1 usa `jakarta.servlet` (Jakarta EE 10); o 9 ainda usa `javax.servlet` e a aplicação não arrancaria.

P. Porque usaram o Maven Wrapper?

R. Para compilar em qualquer portátil sem instalar o Maven: o `mvnw` descarrega a versão certa sozinho.

09

Perguntas de "e se..." (ligação a Sistemas Distribuídos)

PERGUNTA	RESPOSTA CURTA
E se houvesse dois Tomcats?	Cada um teria o seu stock em memória e o `synchronized` só protege dentro de uma JVM → venderiam o mesmo saco duas vezes. Seria preciso estado partilhado (base de dados com transacções, ou um serviço central), sessões partilhadas ou *sticky sessions* no balanceador, e o CSRF e as tentativas de login também teriam de ser partilhados. **Preparem bem esta.**
E se o servidor desligar sem aviso (falha de energia)?	Perde-se o que mudou desde a última gravação (o encerramento ordenado não acontece). O ficheiro nunca fica corrompido (escrita atómica + .bak). Melhoria: gravar periodicamente ou após cada venda.
Porque BigDecimal e não double?	double tem erros de arredondamento (0,1 + 0,2 ≠ 0,3). BigDecimal é exacto — essencial em dinheiro.
Se apagar um produto, as facturas antigas mudam?	Não: a `VendaItem` guarda nome, unidade e preço do momento. O mesmo para os dados do cliente na factura.
Um só trinco para tudo não é lento?	Operações sem relação esperam umas pelas outras, mas cada uma demora microssegundos. Para um agricultor é mais do que suficiente; a simplicidade evita *deadlocks*.
Porque não anular apagando a venda?	Numa contabilidade as facturas não se apagam: a numeração tem de ser contínua. Marcamos como anulada e repomos o stock.
E se o cliente mudar o endereço depois de comprar?	As facturas já emitidas mantêm o endereço da altura (cópia na venda); as novas usam o novo.

10

Guião da demonstração ao vivo (≈ 10 minutos)

Comecem com **dados limpos** (ver secção 13). Em itálico, o que dizer.

Cliente

1. Abrir `localhost:8081/agrogest` → **Criar conta**. Escrever telefone `12345` e submeter. *"Validação no servidor: a mensagem aparece junto ao campo e os outros dados mantêm-se."* Corrigir para `84 123 4567`, província Nampula → entra no catálogo.
2. Pesquisar "feijao" (sem acento). Mostrar o "máx. N" por baixo de cada produto. *"O campo tem como máximo o stock disponível."*
3. Adicionar **3 kg de milho** e **1 saco de adubo**. Tentar adicionar 99999 de enxada → mensagem "Indisponível". *"RN-a: nunca se vende mais do que o stock."*
4. Carrinho: alterar quantidade, remover uma linha e voltar a pôr. Finalizar → Confirmar. *"RN-b: valida tudo primeiro e desconta tudo de uma vez."*
5. Factura **FT-2026-1**: subtotal 2.605,00 + IVA 416,80 = **3.021,80 MT**. Imprimir (pré-visualização). Minhas compras.
6. Mudar o `?id=` da factura para 1 → **403**. Escrever `/admin` → **403**. *"RN-c e RN-d."*

Administrador

7. Sair, entrar como admin. Painel: contagens, stock baixo, validade.
8. Criar o fornecedor **Sementes do Sul, Lda** (SEMENTES, Maputo Cidade). Criar o produto **Semente Feijão nhemba** (320 MT, 400 kg, validade +8 meses). Mostrar que a lista de fornecedores muda com a categoria; tentar FERRAMENTA com esse fornecedor → recusado.
9. Campanha **2026/2027** (01/10/2026 → 30/06/2027). Cultivo **Feijão nhemba, Macia, 8 ha, 15/11/2026** (mostrar as variedades sugeridas; tentar plantio em 09/2026 → recusado). Colheita **3200 kg × 22 MT = 70.400,00 MT**.
10. Tentar remover a campanha → recusado (tem cultivos). Listagens ordenadas, compras detalhadas, relatórios.
11. **Anular** a FT-2026-1 → o stock volta, carimbo ANULADA. *"As facturas não se apagam, anulam-se."*
12. Persistência → Gravar agora; mostrar o `agrogest.ser`. Parar e arrancar o Tomcat → os dados continuam. *"Escrita atómica com .tmp e .bak."*
13. Abrir `VendaService.registarVenda` e explicar as duas fases. Correr `mvnw test` → 37 testes verdes.

11

Comportamentos a conhecer

- Os dados de exemplo só são criados se **não existir** ficheiro. Depois disso, o que aparece é o que está no `.ser`.
- O ficheiro fica em `<pasta do Tomcat>/data/agrogest.ser`, não na pasta do projecto.
- O admin vê o catálogo ("Ver loja") só para consulta e recebe 403 no carrinho, compra e perfil.
- O stock é verificado no formulário, ao adicionar e na confirmação; se mudar entretanto, o carrinho mantém-se e oferece "Ajustar ao stock".
- Um produto apagado sai dos carrinhos, mas as facturas antigas mantêm-se.
- Facturação, relatórios e "total gasto" ignoram vendas anuladas; as listagens mostram-nas com a etiqueta "Anulada".
- Validade é opcional (ferramentas, maquinaria). Ao editar um produto já expirado, pode guardar sem mudar a data.
- Login: 5 falhas bloqueiam esse email durante 1 minuto (o bloqueio vive em memória e reinicia com o Tomcat).
- Se um formulário ficar aberto muito tempo (sessão de 30 min expirou), o envio dá "A página expirou" (login/registo) ou 403 — é o CSRF a funcionar. Basta recarregar.
- O modo escuro fica guardado no browser, não na conta.
- As datas e a numeração FT-ano dependem do relógio do computador.

12

Decisões tomadas e pontos fracos

Decisões (saibam justificar)

Tipo de fornecedor FERRAMENTAS acrescentado	O enunciado tem a categoria de produto FERRAMENTA mas não o tipo de fornecedor. MAQUINARIA pode vender maquinaria e ferramentas.
"Volume" nos relatórios = valor total gasto	Interpretação mais útil para o agricultor; o n.º de compras está no ranking de assiduidade.
Vendas de exemplo em 2025	Para a 1.ª compra da demo ser FT-2026-1, como no enunciado.
Palavra-passe mínima de 4	O exemplo do enunciado usa "1234". Há indicador de força para incentivar melhores.
Só quantidades inteiras	O enunciado usa "3 kg" e "1 saco"; evita problemas de arredondamento no stock.
Colheitas futuras permitidas (até 6 meses após a campanha)	O exemplo do enunciado regista uma colheita em 2027.
Sobreposição de campanhas avisa, não bloqueia	Duas campanhas (ex.: sequeiro e hortícolas) podem decorrer ao mesmo tempo.
Anular em vez de apagar vendas	A numeração das facturas tem de ser contínua.

Pontos fracos (digam-nos vocês antes do docente)

ESCALA Uma só JVM	Dados em memória e `synchronized` não funcionam com vários servidores (ver "e se houvesse dois Tomcats").

DADOS Gravação só no encerramento/manual	Uma falha de energia perde as alterações desde a última gravação. Melhoria: gravar após cada venda ou periodicamente.
CONCORRÊNCIA Um único trinco	Simples e seguro, mas serializa todas as escritas.
FICHEIRO .ser binário	Não se edita à mão; mudanças grandes nas classes podem impedir a leitura (arranca com dados de exemplo).
SEM HTTPS	Em produção, o Tomcat deveria estar atrás de HTTPS (ex.: Nginx ou Cloudflare Tunnel) para proteger as palavras-passe em trânsito.

Já não são pontos fracos

CSRF (agora há token), tentativas de login ilimitadas (agora há bloqueio), venda de produtos expirados, quantidades acima do stock no formulário, admin a comprar, ausência de pesquisa/paginação, impossibilidade de corrigir fornecedores e campanhas.

13

Lista para o dia

☐ Instalar e ensaiar **no portátil da apresentação** (seguir o Tutorial).

☐ Levar o `agrogest.war` já compilado (se o antivírus der erro PKIX, não é preciso compilar).

☐ Começar com dados limpos: Tomcat parado → apagar `data/agrogest.ser` e `.bak` → arrancar.

☐ Abrir sempre `http://localhost:8081/agrogest` (outros sites/portas não têm estas contas).

☐ Ter o código aberto no editor em `VendaService`, `AuthFilter`, `CsrfFilter` e `Persistencia`.

☐ Ter um terminal pronto para `mvnw test`.

☐ Experimentar o `deploy/instalar.sh` numa VM Ubuntu (vale 5% e nunca foi testado numa máquina real).

☐ Cada membro do grupo sabe explicar sem ler: venda atómica, AuthFilter + CsrfFilter, persistência atómica, e "e se houvesse dois Tomcats".

☐ Entregáveis: código (GitHub), WAR, README, **Manual de utilização (PDF)**, Tutorial de instalação (PDF).

☐ Sem Internet, os tipos de letra mudam para os do sistema — tudo o resto funciona.

Contas

Administrador	`admin@agro.mz` / `admin234`
Clientes de exemplo	`ana@`, `carlos@`, `fatima@`, `pedro@`, `rosa@teste.mz` / `1234`
Cliente da demo	`joao@teste.mz` / `1234` (criado durante a apresentação)